Duplicate Finder

How it was built, phase by phase

How I went from a 150-line script to a Windows desktop application with three functions (exact duplicates, similar photos and gathering images), a two-level interface in two languages, a history to undo any operation and a layered architecture checked by tests.

Desktop application for Windows · Python + CustomTkinter
Document generated on 2026-10-24

Contents

1. Starting point . . . 3

1.1. The principles I set for the whole app . . . 3

2. Technology stack and decisions . . . 4

2.1. Architecture: layers that only depend inwards . . . 4

3. Development phases . . . 4

Phase 1 — Core without interface . . . 4

Phase 2 — Main window and scan . . . 5

Phase 2b — Filter by file type . . . 5

Phase 3 — Deciding what to keep . . . 5

Phase 4 — Moving to quarantine . . . 5

Phase 5 — Packaging . . . 5

Extension — Visual redesign . . . 6

Extension — Similar photos . . . 6

Extension — Preview and viewer . . . 6

Extension — Simple / Advanced view . . . 6

Extension — Gather images and history . . . 6

4. Tests . . . 7

5. Problems I found and how they were solved . . . 8

6. Final state . . . 9

7. Quality and publication . . . 9

8. From prototype to architecture: English, layers and languages . . . 10

8.1. How I worked . . . 10

8.2. First part: English, data and languages . . . 10

8.3. Second part: the layers . . . 10

9. What I learned . . . 11

10. What I found while testing it . . . 11

1. Starting point

I have always kept backups of my old devices, such as phones or computers, on external hard drives of several terabytes. Over time, many files end up duplicated in subfolders, and that ends up eating the disk space. Checking by hand is tedious, and it even gets confusing once you no longer know which folders you have already reviewed.

My starting idea was to run quick, simple commands that compare images and other kinds of files and tell me right away whether there were duplicates. I found that the fastest way to do it was to use **Python**.

I started with a simple script of about 150 lines, **buscar_duplicados.py** (“find duplicates”): I typed in a folder path, it walked the folder recursively, computed a SHA-256 hash of the content of every file and told me whether there were duplicate files and how many it had analysed, and also wrote a CSV with the identical groups. It was safe (it only read), but after using it a few times I realised it was impractical: typing the path every time was slow and it did not help me decide what to do with the duplicates.

So I decided to build an **interface** to make it easier to use. While building it I saw that I could add features that fitted the search better to what I wanted, and that is how it went from a script to a **complete desktop program** for Windows, with several features and filtering options, packaged as a single executable that does not require installing Python, and keeping the safety philosophy of the original script.

Goal of the project. On top of solving that need, this project is a **learning exercise**: I built it to go deeper into Python starting from a **basic level**, tackling a real problem from start to finish (algorithms, threads, graphical interface, tests, continuous integration and packaging). I did it with Claude as a programming copilot, and this document and the code guide exist so that one can learn from what was built.

1.1. The principles I set for the whole app

- **An original file is never deleted or modified.** The app only reads and, when the user confirms, *moves* files to a separate folder.
- **Everything is local.** No internet connection, no server, no database. What is saved (settings and history) goes to the user's folder in %APPDATA%.
- **Reversible.** Every operation that moves or copies files leaves a CSV log and an entry in the history, and can be undone.

2. Technology stack and decisions

Element	Choice	Reason
Language	Python 3.12	Reuses the tested logic of the script; quick to iterate.
Interface	CustomTkinter 5.2	Modern look on top of Tkinter, 100% local, light, easy to package.
Content hash	BLAKE2b (standard library)	Faster than SHA-256 and more than reliable enough to detect identical files.
Similar photos	dHash + colour signature (with Pillow)	No heavy dependencies (no OpenCV, no neural networks).
Packaging	PyInstaller 6	Produces a single console-less .exe that runs without Python installed.

2.1. Architecture: layers that only depend inwards

From the start I separated the **logic** from the **interface**: the logic imports nothing from the window and can be tested entirely from the console. Over time that idea became a layered architecture (a pragmatic take on Clean Architecture) inside the package `src/duplicate_finder/`. Dependencies only point inwards, and an automatic test watches over it (see chapter 8).

Layer	Responsibility
domain/	pure rules and data, no input or output: filters, duplicate groups, choosing what to keep, the similarity algorithm (BK-tree, Union-Find).
application/	one use case per class (scan, find similar photos, move to quarantine, gather, history, report) and the **ports** they need: file system, hasher, image fingerprints, history and settings.
infrastructure/	the adapters that implement the ports: local disk (with retries when Windows locks a file), BLAKE2b, Pillow, JSON, error log.
presentation/	the CustomTkinter interface: main window, one component per tab, reusable panels and the background task.
i18n/	the English and Spanish text catalogs.
bootstrap.py	composition root: the only place that builds the adapters and injects them into the use cases.

3. Development phases

I developed the project following a plan in phases. Each phase was tested in isolation before moving on to the next.

Phase 1 — Core without interface

- I moved the scanning logic into its own module, with no interface.
- I changed the hash to **BLAKE2b** with a two-level strategy: first a hash of the first 64 KB (fast, rules out almost everything), and only if it matches is the hash of the whole file computed.

- I organised the scan in three passes: (1) group by size in bytes — a unique size cannot have duplicates; (2) partial hash; (3) full hash. Only the last pass reads whole files.

Note. To make sure the new core behaves like the original script, I added the test `compare_with_original.py`, which builds a tree of folders full of traps (same size and different content, files that only differ at the end, empty files...) and compares the groups detected by both. Result: identical.

Phase 2 — Main window and scan

- I added the folder selector, a two-stage progress bar and a results table.
- The scan runs in a **separate thread** so the window does not freeze; communication between threads goes through a queue that the interface checks periodically.
- The table shows each group of duplicates as an expandable node with its files inside.

Phase 2b — Filter by file type

I wanted to be able to discard files I am not interested in (temporary files, useless extensions...), so I added to the “Scan options” panel the possibility of **searching only certain types** (category checkboxes: Images, Audio, Video, Documents, Archives, which fill in a list of extensions) and also of **ignoring** junk (temporary files, `node_modules`, `Thumbs.db`, 0-byte files...). What does not pass the filter is neither read nor hashed, so it also speeds up the scan.

Phase 3 — Deciding what to keep

- Every group starts with all its copies set to “keep” (nothing would be moved).
- **Automatic rule by preferred folder**: you give a folder and, in each group, if there is a copy inside it that one is kept and the others are marked to move.
- **Validation**: the quarantine button stays locked until every group has exactly one copy marked “keep”.

Phase 4 — Moving to quarantine

- You choose a quarantine folder (it must be outside the scanned folder; the app checks it).
- The marked files are **moved** preserving their subfolder structure, so they can be located and returned if needed.
- If a file with that name already exists in the destination, the new one is saved as `name (1).ext` — nothing is ever overwritten.
- When it finishes, a CSV with every move is written.

Phase 5 — Packaging

- PyInstaller produces `dist/duplicados.exe`: a single console-less file that runs on Windows without Python.
- The app icon is a magnifying glass **drawn by code** (no image files) and converted to a multi-size `icon.ico` with the standard library.

Extension — Visual redesign

I wanted the app to be more attractive, so I redid the interface: a header with a title, numbered sections with icons (Unicode glyphs, no images and no size cost), cards, a statistics panel with the scan result, and an illustrated “empty state” instead of a blank table. The whole colour palette was adjusted to meet the minimum accessibility contrast.

Extension — Similar photos

I wondered whether “relative” photos could be grouped: the same photo at lower quality, with a filter... I added a second tab that detects the **same photo saved several times**: recompressed, at a lower resolution or with a colour filter. For each image it computes a **64-bit dHash** (it measures structure, very stable against recompression and filters) and a **colour signature**. A BK-tree looks for neighbours by Hamming distance without comparing everything with everything, and a Union-Find joins the chains of similarities. I chose **not** to detect crops or rotations, because that would need a much heavier vision engine.

Extension — Preview and viewer

To decide safely I wanted to see what I was about to move:

- A thumbnail on every table row (the real image or a box with the extension, coloured by category).
- A large preview panel on the right, with name, path, size, date and resolution.
- Clicking the preview opens a **near-full-screen viewer** with zoom (mouse wheel) and panning by dragging.

Extension — Simple / Advanced view

Since there were already many options, I decided the app should start in a simplified version, with a switch in the header. It starts in **Simple** (only choose a folder, scan and move; the app preselects the main copy of each group by itself) and remembers the choice. In **Advanced** everything appears: scan options, preferred folder, export CSV, sensitivity slider, history...

Extension — Gather images and history

- I added a third tab that gathers into a common folder the photos scattered across a folder. “By files” strategy (preserving subfolders) or “whole folder”; move or copy.
- A dialog to decide what to do with the files that are not images: leave them, move them, or group them by type in a folder with the name the user chooses.
- **History** (button in the header, Advanced view): lists every move/copy operation and lets you **undo** each one — for a move it returns the files; for a copy it deletes the copies after confirmation.

4. Tests

I test the project with `pytest`, at several levels: unit tests, use cases against in-memory fakes, architecture rules, data migration, internationalisation, and window tests that drive the real interface. File-system behaviour is not “faked”: every test builds real files in a temporary folder, runs the real logic and checks the result on disk. Only the use cases are additionally tested with in-memory fakes of their ports (`tests/fakes.py`), to check their own logic without touching the disk.

Test	What it checks
compare_with_original	The core finds the same duplicates as the original script.
check_filters	Whitelist by category + blacklist + breakdown by extension.
check_selection	Folder rule, manual adjustment, preselection of the main copy.
check_quarantine	Really moving, CSV log, collisions, invalid cases.
check_quarantine_folders	FILES are moved, never folders.
check_perceptual	Grouping of similar photos; a corrupt one breaks nothing.
check_gather	By-files / whole-folder strategies; rules by type.
check_history	Record, list and undo (move and copy).
smoke_window / smoke_similar / smoke_simple	Building the window and end-to-end flows.
smoke_quarantine_ui / smoke_gather_ui	Pressing the real buttons, with simulated dialogs.
smoke_logging / smoke_dialogs	Unexpected errors in the log; viewer, history, dialogs, CSV and thumbnails.
test_contrast	WCAG AA contrast of every text and background pair, in both themes.
test_use_cases	Every use case with in-memory fakes of its ports.
test_architecture	Which layer may import which; the domain and the use cases do not touch the disk.
test_i18n	English and Spanish have the same keys and placeholders; every text the code asks for exists.
test_data_migration	Data saved by version 1.0 is read and converted without modifying the original.
test_background_job	The background task: progress, cancellation and failures.
test_scanner_passes / test_logging / test_units	Every scanner pass, the error log, the settings, the CSV and the version.

5. Problems I found and how they were solved

Problem	Solution
CustomTkinter re-applied its own icon 200 ms after the window opened.	Set the icon with iconbitmap(.ico) instead of iconphoto, so the library's own internal check detects it and does not overwrite it.
Passing several images at once to iconphoto() blurred the icon.	Use a single well-chosen image, or the multi-size .ico in the .exe.
The resolution of photos came out wrong in the viewer and the list.	Read im.size BEFORE im.draft(): draft shrinks the image and changes the reported size.
The “empty state” did not cover the gallery (CTkScrollableFrame).	Toggle with grid()/grid_remove() instead of tkraise().
Pillow inflated the .exe from 13 to 20 MB.	I accepted the cost when adding similar photos and thumbnails; Pillow is necessary and goes in requirements.txt (reportlab does not, since it only generates this documentation).
A doubt of mine while testing: “does it move folders instead of photos?”.	Two new tests confirmed that individual files are always moved; what you see is the structure recreated at the destination and the source subfolders left empty.
After moving to quarantine, the preview kept the previous image under the “select a file” text.	CTkLabel.configure(image=None) does not remove it: the inner tkinter Label has to be cleared.
With many similar photos, the gallery got cut off and the scrollbar vanished.	Windows does not draw windows taller than 32,767 px. Now the groups are split into pages (Previous / Next) and a test checks that none gets near that limit.
My window tests wrote to the app's real history.	They now use a temporary APPDATA, and a shared check warns if any of them touches the real data again.
Two pollers reading the same queue stole each other's messages.	Separate queues for thumbnails and for the operation in progress.
When undoing a copy, Windows sometimes still had the file in use (antivirus, indexer) and the deletion failed.	The deletion retries for a moment before reporting a failure, with tests that simulate it.

6. Final state

Metric	Value
Application code (src/)	~7,200 lines in 51 modules
· domain/	~800 lines, 8 modules
· application/	~1,200 lines, 13 modules
· infrastructure/	~500 lines, 8 modules
· presentation/	~3,700 lines, 18 modules
· i18n/ (with both catalogs)	~1,000 lines
· tests (tests/)	~4,300 lines
Application dependencies	customtkinter, pillow
Final executable	duplicados.exe (~20 MB, single file) or ZIP with a folder (~20 MB), no console
Quality	tests green in CI on every change; required minimum coverage: 90 % in the logic layers and 70 % in the interface

The application covers three related needs — finding exact duplicates, finding similar photos and gathering scattered images — and in all three I kept the same rule: it never deletes your originals and everything can be undone.

7. Quality and publication

Before publishing the project on GitHub I spent a round of work on quality, always with the tests as a safety net:

- **Structure.** The main window had more than 2,100 lines in a single class. Now it is a shell of a little over 200 lines and each tab is a component with its own state, in its own module (`exact_duplicates_tab.py`, `similar_photos_tab.py`, `gather_images_tab.py`). I did it tab by tab, checking each time that the 16 screenshots of the interface (light and dark theme) were still pixel-identical. I also split the scan into one function per pass.
- **Tooling.** pytest (with coverage of the logic and the interface), ruff for style and mypy for the types of the logic layers, which are fully annotated. Everything runs on every push with GitHub Actions, and Dependabot warns about updates.
- **Contrast.** The tests measure every text and background pair with the WCAG formula in the light and dark themes (AA level, 4.5:1). With them I replaced CustomTkinter's default blues and greys, which did not reach it with white text, by colours of my own (5.7:1).
- **Error log.** Without a console, the .exe's errors vanished. Now they are written to a file and the app tells you where it is.
- **Performance.** `tests/benchmark.py` compares the 3-pass scan with hashing everything: it reads 66 % fewer bytes and finds exactly the same groups.
- **Two download formats.** With Avast installed, the single-file .exe froze on first start (it unpacks into a temporary folder every time). That is why every version also includes a ZIP with a folder,

which extracts nothing at startup. While testing it I saw that, without a console, a mere CustomTkinter warning (a path longer than 260 characters) made the app crash; now outputs that do not exist are discarded.

- **Privacy in screenshots.** The screenshots and the GIF are made on a sample folder with a neutral path, so the Windows user name does not show.

How I did it. I developed it with Claude (Anthropic) as a programming copilot: I decided what to build and reviewed the result, and Claude wrote and tested the code in joint working sessions. The commits carry the “Co-Authored-By” line to make that clear.

8. From prototype to architecture: English, layers and languages

With version 1.0 published, I decided to make the project more polished and uniform: all the code in English, a layered architecture, and an interface available in two languages, with the documentation in Spanish and in English too. I did it as a series of small changes, each with its own branch, its own *pull request* and a green CI before accepting it.

8.1. How I worked

- **Two permanent branches.** `main` is the published version and `develop` is where the work is integrated. Every change branches from `develop` with a name that describes the change, and comes back through a *pull request* that is merged (*squash*) only with a green CI and my approval.
- **Commit messages** in the Conventional Commits format (`refactor:`, `test:`, `docs:`...), in English.
- **Verification at every step.** Linter, types, the whole test suite with its minimum coverage, the 16 screenshots of the interface compared pixel by pixel with those from before the change, and the .exe really built and launched (folder and single-file).

8.2. First part: English, data and languages

- **Identifiers in English.** Modules, classes, functions and variables (`pruebas/` became `tests/`). With a tool that renames by *tokens* and checks that the syntax tree of the code does not change; the strings that name code (lists of modules, `getattr`...) had to be reviewed by hand.
- **Comments and docstrings in English** (more than 600), checking that the code is identical except for the docstrings.
- **Saved data in English, without breaking anything.** The keys of `history.json` and `config.json` moved to English; on startup the files from version 1.0 are read and converted without modifying them (a backup copy is kept).
- **Text catalog.** Every interface text moved into a catalog (`tr("key")`), one per language. The app starts in the language of Windows, has a selector in the header, and a test checks that both catalogs have the same keys and placeholders.
- **Tests, CI and templates** in English too.

8.3. Second part: the layers

- **Package and layers.** The code moved to `src/duplicate_finder/` with the folders `domain`, `application`, `infrastructure` and `presentation`. The modules that mixed logic with input/output were split.
- **Ports and adapters.** The use cases ask for the disk, the hasher, the image fingerprints, the history and the settings through interfaces (`application/ports.py`); `infrastructure/` implements them and `bootstrap.py` is the only place that connects them. The domain stopped reading the disk.
- **Use cases that hold the logic.** The three tabs repeated the same block (validate, confirm, move, record in the history). Now the use case does it, and the tab only shows and asks.
- **Smaller tabs.** The biggest tab (900 lines) was split into components (options panel, preview), and the "thread + queue + polling" pattern that we wrote by hand four times ended up in a single class, `BackgroundJob`.
- **The rules, as a test.** `tests/test_architecture.py` reads every module with `ast` and fails if a layer imports what it should not, or if the domain or the use cases touch the disk. The checker itself is tested with snippets that break each rule.

Note. None of this changed what the user sees: the screenshots of the Spanish interface came out pixel-identical before and after every step of the second part.

9. What I learned

- **What was hardest:** how to split the code so that no file ended up too long and each part had a single job. The first version separated the logic from the interface; later I organised it in layers with ports and adapters, and split the biggest tabs into components.
- **What I learned most:** the versatility of Python for running scripts directly on files: with very few lines you can walk through a disk, compare contents and move files. And that a rule written as a test holds far better than one written in a document.
- **Changing with a net.** With tests, compared screenshots and small changes, a whole project can be reorganised without fear of breaking it.

There is no fixed roadmap: I will keep improving the application as I go, depending on which features I find it lacks in practice. If anyone has ideas for future features, I would love to hear them.

10. What I found while testing it

I tested the app myself, with my own files, and these are the problems I found that changed the program:

- **The .exe would not start.** When I downloaded the executable from the release I got the error "Failed to start embedded python interpreter". It was my antivirus (Avast), which holds the single file for a few minutes the first time it is opened. That is why every version also includes a ZIP with a folder, which does not have this problem.
- **The "Similar photos" gallery got cut off.** With many photos, some images appeared cut off and on the right there was a gap that looked like a scrollbar but was not one. It turned out to be a Windows limit on very tall windows, and now the groups are split into pages.

- **“Gather images” moved folders instead of photos.** I noticed it while testing and it was fixed; an automated test now checks it.
- **My phones did not show up.** I asked whether the app could read a phone connected by USB. It cannot: Windows does not give them a normal folder path. It is noted in the limitations, with the workaround of copying the photos to the PC first.
- **The screenshots must not show my Windows username.** I asked for the images and the GIF in the README to use a sample folder instead of a path with my name.